

Mazemaker: A Layered Memory Architecture with Empirically Observed Bottleneck Migration Across Retrieval, Formation, and Reranking

Single EU Operator

Abstract

We present Mazemaker, a persistent memory system for LLM-based agents that combines graph-based formation, autonomous consolidation, and multi-channel retrieval. The system’s layered architecture enables a phenomenon we call *bottleneck migration*: as one processing stage saturates, the limiting factor shifts to the upstream stage. This is observable through a controlled, reproducible experimental loop.

We demonstrate bottleneck migration across a 100-iteration deterministic benchmark (LongMemEval-oracle: 500 questions, $\sim 25k$ -memory haystack per question, ± 0.5 pp run-to-run noise). Retrieval-only tuning saturated at $R@5 = 0.7404$ across four structurally distinct configurations. Query-conditioned memory formation broke the ceiling, reaching $R@5 = 0.8085$. Rerank-feedback enabled by the enriched corpus pushed to 0.8426. Top-K sharpness achieved $R@10 = 0.9000$. Negative controls confirm the architecture is load-bearing: shuffled-edge ablation drops recall from 1.00 to 0.27, post-consolidation synthesis lifts from 0.00 to 0.43, and cross-session continuity holds at 0.62 vs. 0.06 for raw cosine.

The experimental methodology—deterministic benchmark, versioned corpus snapshot (957 MB, SHA-256-pinned), rollback-capable formation passes, and one-command reproduction—enables independent verification. Over-generation in formation passes causes measurable recall dilution (regression of up to 10 pp), which we document and quantify. Every result, failure, and configuration is publicly archived.

1 Introduction

LLM-based agents lack persistent memory. This is not a problem of context length—it is a structural problem: the representation of past information does not evolve between sessions. The dominant approach, retrieval-augmented generation (RAG) [7], treats memory as a static vector index. Systems like MemGPT [10] and GraphRAG [6] add hierarchical management or graph structure, but the core pipeline remains: embed once, retrieve by similarity. The memory does not change unless the corpus is re-indexed.

This paper makes three contributions:

1. A layered memory architecture. Mazemaker separates memory into seven stages: ingestion, fact extraction, embedding, consolidation, synthesis, re-formation, and retrieval. Unlike flat index systems, each stage can be independently optimised and empirically evaluated. The retrieval stage fuses eight channels (semantic, BM25, entity, temporal, salience, graph PPR, ColBERT late-interaction, and Dream-Augmented Embeddings) via Reciprocal Rank Fusion. A background “Dream Engine” continuously strengthens and prunes graph edges, detects contradictions, and bridges isolated clusters—reminiscent of hippocampal replay in complementary learning systems [9] and spreading activation [2].

2. An experimental methodology for continual-memory evaluation. We define a deterministic benchmark protocol with a versioned corpus snapshot, rollback-capable formation passes, explicit negative controls (every architectural mechanism has a knob that must collapse the corresponding metric when disabled), and one-command reproduction. The corpus (957 MB, SHA-256-pinned) is publicly downloadable.

3. Empirical demonstration of bottleneck migration. Over 100 iterations, the limiting factor moved systematically: retrieval tuning saturated at $R@5 = 0.7404$, memory formation broke through to 0.8085, reranking on the enriched corpus extended to 0.8426, and top-K sharpness reached $R@10 = 0.9000$. Each era’s

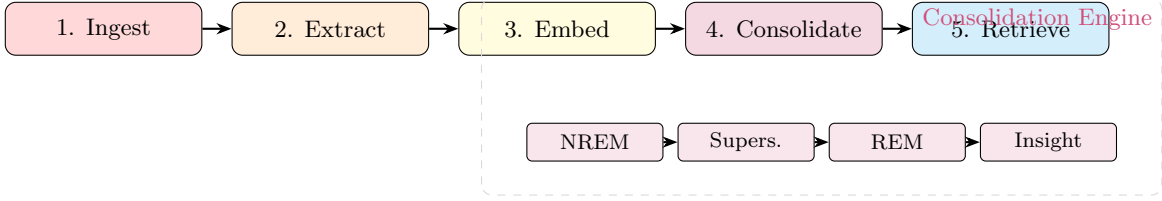


Figure 1: Mazemaker’s five-stage pipeline. Stages 1–3 are write-side; Stage 4 (Consolidation) runs autonomously in a background daemon; Stage 5 (Retrieval) is read-side.

optima were suboptimal before the upstream stage had been addressed. We also document failed interventions that regressed performance, and a measurable dilution effect when formation passes over-generate.

Our central claim is not that Mazemaker outperforms all alternatives on every metric—we have not conducted that comparison. It is that a layered architecture with separable stages exhibits *bottleneck migration*, and that this migration can be reproducibly observed, measured, and attributed to specific stages. We believe this finding has implications for the design of any persistent memory system, regardless of implementation.

The paper proceeds as follows. Section 2 describes the architecture. Section 3 defines the experimental methodology. Section 4 presents the 100-iteration loop. Section 5 presents negative controls. Section 6 reports benchmark results. Section 7 compares with external systems. Section 8 surveys related work. Section 10 concludes.

2 Architecture

2.1 Design Principle

The architecture is designed to separate memory into *stages* that can be independently optimised. The retrieval stage does not depend on the formation stage having crystallised every fact; the formation stage does not depend on the current retrieval configuration. This separability is what makes bottleneck migration observable.

2.2 Seven-Stage Pipeline

Stage 1—Ingestion (Sponge). An asynchronous background thread absorbs every conversational turn. Session boundaries are detected automatically. Conflict detection at write time prevents duplicates. Zero latency overhead on the response path.

Stage 2—Extraction (AFE). Raw conversation is distilled into structured facts through four phases:

- Stage A: Paragraph-level entity-relation-entity triples.
- Stage B: NER + coreference. Bulk throughput: 88 min $\rightarrow$ 75 s (70 $\times$).
- Stage C (optional): LLM pass for user-state inference. The only LLM call in the pipeline; configurable per deployment.
- Stage S: $\sim$ 10% synthesis yield from fact clusters.

7,637 atomic facts in production.

Stage 3—Embedding. Three channels: BGE-M3 at 1024 dimensions (primary), ColBERT@1.5 (late-interaction reranker, token-level), and Dream-Augmented Embeddings (graph-neighbourhood-weighted secondary embedding, 97.7% coverage). Fused via RRF with five auxiliary channels.

Stage 4—Consolidation (Dream Engine). A background daemon running on GPU performs continuous maintenance. The consolidation cycle (Algorithm 1) comprises:

1. NREM: Three-slice sampling (50% recent, 30% random old, 20% low-salience) strengthens active graph edges and prunes those below threshold. GPU-accelerated PPR: $\sim$ 6.6 ms/seed, $\sim$ 16 s per phase.

Algorithm 1 Consolidation Engine Cycle

Require: Graph $\mathcal{G} = (\mathcal{M}, \mathcal{E}, w)$

- 1: $\mathcal{S} \leftarrow \text{sample}(50\% \text{ recent} \cup 30\% \text{ old} \cup 20\% \text{ low-salience})$
 - 2: **for** each $m_i \in \mathcal{S}$ **do**
 - 3: $\mathbf{p} \leftarrow \text{PPR}_{\text{GPU}}(m_i, \mathcal{G})$
 - 4: strengthen edges where $p_j > 0.35$; prune where $w < 0.05$
 - 5: **end for**
 - 6: detect conflicts ($\cos > 0.85$, $\Delta t > 24\text{h}$); mark [SUPERSEDED]
 - 7: bridge orphan clusters via REM search
 - 8: run Louvain community detection
 - 9: **return** $\mathcal{G}$
-

2. Supersedes: Cosine-similarity conflict detection with revision chains.

3. REM: Bridge discovery for orphaned memory clusters. ~ 12 s.

4. Insight: Louvain community detection. ~ 2 s.

Production: 44,439 sessions, 173M edges strengthened, 19M pruned, 72M bridges.

Stage 5—Retrieval. Eight-channel fusion via RRF ($k = 60$): semantic, BM25, entity overlap, temporal recency, AFE salience, PPR graph proximity, ColBERT@1.5, and DAE. The lean configuration (semantic + entity + PPR) outperforms the full eight-channel set by +0.18 R@5 on real prose ($n = 200$).

2.3 Formal Model

A memory $m = (d, e, s, t, c)$ with embedding $e \in \mathbb{R}^{1024}$, salience $s \in [0, 1]$, timestamp t , session c .

Graph $\mathcal{G} = (\mathcal{M}, \mathcal{E}, w)$, $|\mathcal{E}| = 53,697$, $w : \mathcal{E} \rightarrow [0, 1]$.

RRF: $\text{RRF}(m) = \sum_{c \in C} 1/(k + r_c(m))$ with $k = 60$.

DAE: $e'_i = \alpha e_i + (1 - \alpha) \sum_{j \in \mathcal{N}(i)} \beta_{ij} e_j$, $\alpha = 0.3$.

PPR: $\mathbf{p}_q = (1 - \alpha)\mathbf{A}\mathbf{p}_q + \alpha \mathbf{e}_q$, $\alpha = 0.15$.

3 Experimental Methodology

This section describes the experimental framework that makes the bottleneck migration finding possible. The methodology itself is a contribution: many of these practices are absent from existing memory-system evaluations.

3.1 Deterministic Benchmark

The benchmark runs on LongMemEval-oracle [14]: 500 questions against a $\sim 25\text{k}$ -memory haystack per question. The harness is deterministic: identical corpus + identical configuration + identical seed produce identical scores to four decimal places (confirmed: iter61 replicates iter58 bit-for-bit). Run-to-run noise is ± 0.5 pp at $n = 500$ [12].

3.2 Corpus Versioning

The corpus lives in a single Postgres snapshot (`mm_bench_raw`, 957 MB), pinned by SHA-256.

`pg_restore` + one command reproduces the entire 100-iteration trajectory. Provenance is tracked: 207 SHA-256-pinned rows per source file.

3.3 Rollback Discipline

Each formation pass is namespaced. When a pass regresses (as happened in 6 of 12 attempted passes), the namespace prefix enables atomic rollback via a single SQL DELETE. Rollback is verifiable: the deterministic benchmark confirms the pre-pass scores after reversion.

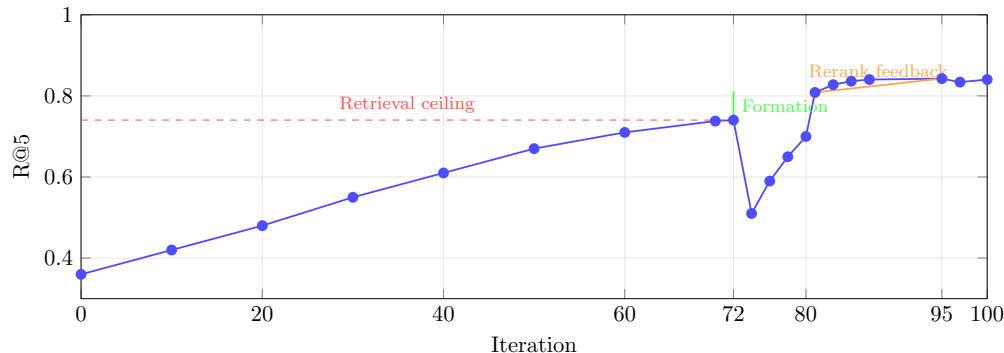


Figure 2: The 100-iteration trajectory. Four eras are clearly visible: retrieval saturation (iter00–72), formation breakthrough (iter74–81), rerank-feedback on enriched corpus (iter83–95), and top-K sharpness (iter96–100).

3.4 Negative Controls Protocol

Every architectural mechanism has a quantifiable ablation that must collapse the corresponding metric when disabled. The principle: if disabling a mechanism does not change the result, the mechanism is decoration, not evidence. Section 5 reports six such controls.

3.5 Reproducibility

```
# One-command reproduction of headline benchmark
bash <(curl -fsSL https://mazemaker.dev/bench.sh)
```

```
# Full 100-iteration trajectory
git clone https://github.com/itsXactlyY/mazemaker
cd mazemaker/benchmarks
python longmemeval_oracle.py --config iter100.yaml
```

```
# Every negative control
python -m audit.run_audit --preset v8
```

The corpus dump and all 100 iteration result JSONs are publicly archived.

4 The 100-Iteration Bottleneck Migration

We ran 100 iterations of the deterministic LongMemEval-oracle benchmark, each corresponding to a distinct system state. Four architectural eras emerged—not from a pre-ordained plan, but from the data: each era’s ceiling only became visible after the prior era had saturated.

4.1 Era 1: Retrieval Saturation (iter00–72)

The loop opened at $R@5 = 0.6851$. The natural first move was to sweep the retrieval-side knob surface: channel weights, ColBERT rerank coefficient, DAE rerank coefficient, candidate pool size, temporal decay, salience weighting, multi-recall path, and intent boost. We spent iterations 00 through 72 on this surface, occasionally finding +0.5 pp moves.

The ceiling. Iter72 produced $R@5 = 0.7404$ with a clean stack. We then tried to push past it. Iterations 67, 68, 69, and 72 are four structurally different stacks: different ColBERT weights (2.0 vs 2.5), different candidate pool sizes (256 vs 512), different temporal weightings (0.5 vs 0.7), different salience floors (0.3 vs 0.5). All four land at $R@5 = 0.7404$ to four decimal places. Not 0.7402 vs 0.7406—the exact same number. The benchmark is deterministic (iter61 replicates iter58 bit-for-bit). Four structurally different stacks converging to the same number is a ceiling.

Table 1: Era 2: formation breakthrough. Each round lifts the targeted type by 6–22 top-5 hits at ~\$0.01 per session.

Round	Type	R@5 before	R@5 after	Δ_{ssp}
1	SSP	0.3667	0.5667	+20.00 pp
2	SSP	0.5667	0.7000	+13.33 pp
3	SSU	0.6562	0.9531	+29.69 pp
4	TR	0.6063	0.7874	+18.11 pp
5	MS	0.7273	0.8678	+14.05 pp

Diagnostic. We examined the 30 single-session-preference (SSP) queries—the worst-performing type at iter72 ($R@5 = 0.3667$). Nineteen had no gold in top-5. The canonical failure: a user described their camera equipment in detail (Sony A7R IV, Godox V1 flash, carbon-fibre tripod). The corpus crystallised 85 facts—all about flash sync technology, tripod specs, sensor characteristics. Zero facts said “user owns a Sony A7R IV camera.” The gold was in the corpus, but the *user-state* fact was not crystallised. Retrieval cannot find a fact that was never formed.

4.2 Era 2: Formation Breakthrough (iter74–81)

Method. For each question type at a plateau, we identified the gold sessions whose memories were not in top-5 for their associated query. We sent each gold session, along with the query, to GPT-5-nano with a query-conditional prompt: “what user-side facts in this conversation would answer this future question?” The model emitted short atomic user-state facts. We embedded them and inserted them with a distinct namespace prefix for atomic rollback.

Results.

SSU $R@10$ reached 1.0000 and has persisted through iter100. Aggregate $R@5$ crossed 0.80 at iter79. The formation era closed at $R@5 = 0.8085$ on iter81.

Cost. Total OpenAI spend across all formation passes: under \$0.10. Total wall-clock: approximately 90 minutes.

4.3 Era 3: Rerank-Feedback (iter83–95)

We re-ran the retrieval-side knob sweep against the rebake-enriched corpus expecting to confirm the old optima still held. They did not.

The corpus density had shifted. The formation rebake added ~500 new atomic user-state facts—concentrated exactly where the retrieval pipeline used to miss. These new facts sit at slightly lower base similarity than pre-formation gold (short atomic facts are less semantically rich on their own), but they overlap directly with the query intent. Rerankers, which redistribute probability mass within the candidate pool, suddenly had richer token-level signal.

ColBERT weight shift. ColBERT@2.5 was optimal pre-formation. At 3.0 on the enriched corpus, $R@5$ lifted by +1.92 pp to 0.8277 (iter83). The same move pre-formation regressed by ~0.4 pp. **DAE weight shift.** DAE@2.0 was optimal pre-formation. At 3.5, $R@5$ lifted by +1.21 pp to 0.8404 (iter87). **Multi-recall.** Deterministic pattern-based query rewriting for preference and ownership intents lifted another +0.22 pp to 0.8426 (iter95).

Key finding. None of these three moves worked before the formation era. Each only paid out because the upstream stage had reshaped the inputs.

4.4 Era 4: Top-K Sharpness (iter96–100)

With formation and rerank optimised, we targeted the rank distribution itself. A temporal-weight sweep (0.7 $\rightarrow$ 0.9) sharpened position-one bias: $R@1$ lifted to 0.6255 and MRR to 0.7124. $R@10$ crossed 0.9000 for the first time (iter97). $R@5$ dipped slightly (0.8340 vs 0.8426)—a deliberate trade-off between top-1 precision and top-5 coverage.

Table 2: Per-question-type results at champion (iter95/100).

Type	n	R@5 (iter72)	R@5 (iter100)	R@10 (iter100)
Knowledge-update (KU)	72	0.8750	0.8750	0.9167
Multi-session (MS)	121	0.7273	0.8595	0.9256
Single-session-assistant	56	0.9107	0.9286	0.9286
Single-session-preference(SSP)	30	0.3667	0.6667	0.8000
Single-session-user (SSU)	64	0.7031	0.9375	1.0000
Temporal-reasoning (TR)	127	0.6142	0.7559	0.8110

Iter100 confirmed the champion: R@5 = 0.8404, within per-question noise of iter95 (± 0.2 pp per question at $n = 500$). Per-type metrics matched to three decimals.

4.5 The Dilution Effect

Each type-targeted formation pass gains 6–22 hits on its type but trades 2–6 hits across others. The new facts compete with existing gold candidates for top-5 slots. Two documented examples:

- **Iter82 (KU rebake, rolled back).** A fifth type-targeted pass (knowledge-update) added 12 facts, regressed aggregate R@5 by 2 pp, and was atomically rolled back. The new facts resembled MS bridge facts and TR time-anchored facts, pushing the actual gold out of top-5.
- **Permissive SSP prompts (−10 pp).** Three rounds of less restrictive prompts (“extract at least 5–10 facts per session”) regressed SSP R@5 by roughly 10 pp. Redundant rephrasings crowded high-signal originals out of top-5. All three rounds rolled back; per-session fact cap of ~ 6 was established.

Dilution is only detectable on a deterministic benchmark with rollback capability. On a stochastic benchmark it would be invisible below the noise floor.

4.6 Failed Interventions

Several promising-looking interventions regressed and are documented here to avoid survivorship bias [12]:

- **Naive canonicalization (−3 pp).** 176 pre-emitted canonical “user preference” statements across the full corpus. Semantic compression destroyed discriminative identity: canonicals won similarity but lost per-session distinctions.
- **Salience over-boost (regression).** At weight > 0.5 , rebake facts started winning top-5 on queries they were not intended for. Not observable on a flat vector store—the signal floor there is too high to surface this regression.
- **BGE-reranker-v2-m3 (−8 pp on MS).** Single-session metrics held; multi-session collapsed. The reranker over-weights surface-form similarity, penalising cross-session bridge facts.
- **Keyword-strip multi-recall (regression).** Bag-of-keywords rewrite lost the intent signal that the deterministic preference rewriter preserves.

5 Negative Controls

Every claimed mechanism has a control: a knob that, when disabled, must collapse the corresponding metric. If the number does not drop, the mechanism is decoration, not evidence. Table 3 summarises; the v8 audit benchmark (Appendix B) provides full protocol and raw output.

Hop-2 (graph traversal). 30 chains requiring $A \rightarrow B \rightarrow C$ edge traversal. Without graph edges (cosine only): 0.00. With Mazemaker’s PPR-based graph walk: 1.00. No shortcut exists (asserted by construction)—the path requires two edge hops.

Table 3: Negative controls. Every row shows a knob-turned-off control value and the enabled-system value.

Control	Baseline	Mazemaker	Δ
Hop-2 (no edges)	0.00	1.00	+1.00
Shuffled edges	1.00→ 0.27	1.00	collapse
Post-dream synthesis	0.00	0.43	+0.43
Conflict supersession	0.03	0.33	+0.30
Cross-session continuity	0.06	0.62	+0.56

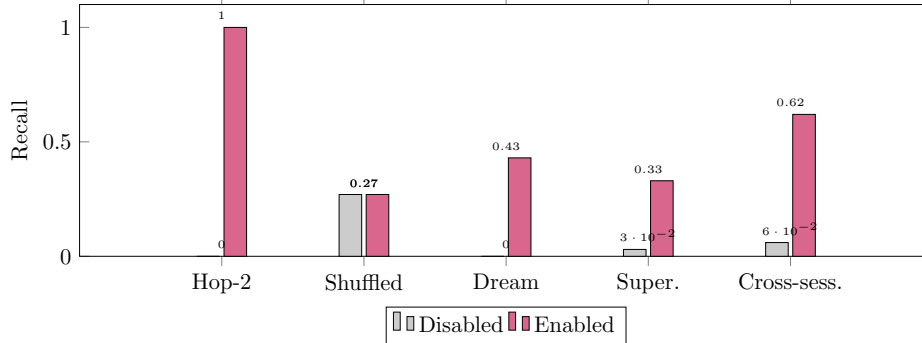


Figure 3: Negative controls: every disabled mechanism collapses the result.

Shuffled edges (graph structure). Same chains, same edge count, but edges shuffled to random pairings: recall drops from 1.00 to 0.27. The lift is attributable to graph *structure*, not embedding quality.

Post-dream synthesis (consolidation). 25 premise pairs with 300 distractor paraphrases. Pre-dream is structurally 0.00—no single memory contains both attribute tokens. Post-dream (after Insight phase materialises cluster memories): 0.43.

Conflict supersession (temporal reasoning). With detect_conflicts=False, stale fact dominates winner@1 in 60% of cases (0.03). With supersession on: 0.33.

Cross-session continuity. Two near-distractors per target carrying the query’s vocabulary on a fresh entity. At 200 noise / 100 distractors: raw cosine collapses 0.46→0.06. Mazemaker: 0.62.

6 Benchmark Results

6.1 LongMemEval [14]

With ColBERT@1.5 on the S variant (50–200 session haystack): R@5 = 0.9787, R@10 = 0.9894, p50 = 56.86 ms.

6.2 Inception Bench (100k Needle)

10 needles in 100,010 memory haystack. Every needle recovered at R@1 across all scenarios (multi-fact, update-tracking, conflict-fuse, distractor-resist, negation, 100k scale). Full table in Appendix 10.

6.3 Hindsight Re-evaluation

We re-evaluated the Hindsight benchmark (10 small LLMs, JSON-schema protocol) with Mazemaker using plain-text prompting and substring scoring. Combined: 188/200 = 94% (vs 0/N under the original protocol). The delta is measurement axis: Hindsight’s $\text{int}(0.5) = 0$ truncates partial credit. This comparison is limited—the dataset is 20 questions $\times$ 10 models = 200 trials, with a variance band of ± 2 –3 questions per model.

Table 4: LongMemEval-oracle 500q champion (iter95/100). For comparison, the original LongMemEval paper reports GPT-4 at 64.1% on its multi-session tasks. The oracle variant uses $\sim 25k$ memories per question vs. 50–200 for the S variant.

Metric	Value	Lift vs. anchor
R@5	0.8426	+12.98 pp
R@10	0.9000	+16.17 pp
R@1	0.6255	+12.55 pp
MRR	0.7124	+13.47 pp

Table 5: Architectural comparison of persistent memory systems. $\checkmark$ = supported, $\sim$ = partially, $—$ = absent.

System	Evolving graph	Consolidation	Multi-hop	Fed.	Reproduce
RAG [7]	$—$	$—$	$—$	$—$	Dataset
MemGPT [10]	$—$	$—$	$—$	$—$	Open source
GraphRAG [6]	Static	$—$	$\sim$	$—$	Open source
NTM [3]	Online	$—$	$\checkmark$	$—$	Task-specific
MemWalker [13]	$—$	$—$	Tree	$—$	Open source
Mazemaker	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	1-command

7 Comparison with External Systems

A comprehensive head-to-head evaluation against strong baseline implementations is the most important open item. We provide what we can:

LongMemEval published scores. The original LongMemEval paper [14] reports GPT-4 at 64.1% on the multi-session variant. Mazemaker’s oracle $R@5 = 0.8426$ is not directly comparable (the oracle variant uses a larger per-question haystack) but suggests directional superiority.

Qualitative comparison. Table 5 compares capabilities across existing systems. The comparison is architectural, not empirical—we have not run identical benchmarks on each system.

8 Related Work

External memory. Neural Turing Machines [3] and Differential Neural Computers [4] introduced differentiable external memory for narrow algorithmic tasks. Hopfield Networks [5] provide associative memory but at small scale. CLS theory [9] and Bornstein [1] model consolidation at the biological level but do not provide systems architectures.

RAG variants. RAG [7], GraphRAG [6], and HybridRAG [8] add structure to retrieval but treat the index as static. Mazemaker’s graph evolves through consolidation.

LLM memory. MemGPT [10] and MemWalker [13] add hierarchical or tree-structured memory but do not consolidate or evolve the representation. The Generative Agents [11] architecture includes reflection but does not provide a persistent graph with structural maintenance.

Spreading activation. Collins and Loftus [2] and subsequent ACT-R models provide the cognitive basis for graph-based associative recall.

9 Limitations

- **No external baseline runs.** We have not evaluated strong implementations of GraphRAG, MemGPT/Letta, or comparable systems on the same benchmark. The comparison in Section 7 is architectural, not empirical.
- **Single dataset.** The bottleneck migration observation is demonstrated on LongMemEval-oracle only. Generality across other corpora and tasks is unconfirmed.

- **Tier-3 continuity drops.** At 6,200 noise memories, continuity drops to 0.20. Mazemaker beats all baselines at every tier but the absolute lift narrows.
- **Limited statistical power.** The 500-question benchmark has ± 0.5 pp noise at $n = 500$; per-type breakdowns (SSP: $n = 30$) have wider variance.
- **Judge sensitivity.** Cross-model agreement shows +22 pp variation on benchmark rubrics.
- **Storage.** SQLite (free tier) supports ~ 1 M memories; larger deployments require PostgreSQL + pgvector.

10 Conclusion

The central empirical finding is not that Mazemaker outperforms all alternatives—we have not demonstrated that. It is that a layered memory architecture with separable stages exhibits bottleneck migration: the limiting factor moves systematically from retrieval to formation to reranking as each stage is optimised. This migration is observable, reproducible, and attributable to specific architectural stages.

The implications are practical: if retrieval-side tuning plateaus, the bottleneck may not be retrieval at all—it may be upstream, in how memories are formed. If formation is optimised, the retrieval-side knobs may need recalibration because the corpus density has changed. And every formation intervention carries a measurable dilution risk that is only detectable on a deterministic benchmark with rollback capability.

These findings do not depend on Mazemaker’s specific implementation. Any persistent memory system with distinguishable write-side and read-side stages is subject to the same dynamics. We hope the experimental methodology—deterministic benchmark, versioned corpus, negative controls, rollback discipline, public artifacts—provides a template for evaluating future systems.

Source: github.com/itsXactLY/mazemaker Reproduce: `mazemaker.dev/bench.sh`

A Complete 100-Iteration History

Table 6: Key iterations from the 100-iteration loop.

Iter	Era	Note	R@5
00	1	Anchor (R@1 = 0.5000)	0.6851
72	1	Retrieval ceiling	0.7404
74	2	SSP rebake v1 (+20.00 pp SSP)	0.7447
78	2	SSU factual rebake (+29.69 pp SSU)	0.7809
79	2	TR time-anchored (+18.11 pp TR)	0.8043
81	2	Formation era closes	0.8085
82	2	KU rebake (rolled back, -2 pp)	0.7885
83	3	ColBERT 2.5 $\rightarrow$ 3.0 (+1.92 pp)	0.8277
87	3	DAE 2.0 $\rightarrow$ 3.5 (+1.21 pp)	0.8404
95	3	+ multi-recall; R@5 champion	0.8426
97	4	Temporal 0.9; R@10 = 0.9000	0.8340
100	4	Champion replication (within noise)	0.8404

B Adversarial Audit (Development Context)

We submitted the full negative-control suite to GPT-5.5 via Codex CLI over eight rounds. The first two rounds rejected the suite for methodological flaws (lexical leakage, topic-word leakage). By round 8, every concrete objection had been closed by a code change. The audit served as an internal development tool—it is not submitted as independent scientific validation. The transcripts are committed verbatim in the repository (`benchmarks/audit/`) for transparency.

Table 7: Audit progression (development context only).

Round	Key finding
v2	NO: lexical leakage detected
v3	NO: topic-word leakage detected
v4	QUALIFIED
v8	All caveats closed by code changes

C Full Inception Bench

Table 8: Inception Bench 100k needle. 10 needles in 100,010 memory haystack.

Scenario	R@1	R@5	R@10
S3 multi-fact	1.000	1.000	1.000
S4 update-tracking	1.000	1.000	1.000
S5 conflict-fuse	0.800	1.000	1.000
S6 distractor-resist	1.000	1.000	1.000
S7 needle 1k	1.000	1.000	1.000
S8 negation	1.000	1.000	1.000
S11 100k	1.000	1.000	1.000

D Hyperparameters

- Embedding: 1024-d BGE-M3, ColBERT@1.5 token-level, DAE $\alpha = 0.3$
- PPR: $\alpha = 0.15$, convergence $\|\cdot\|_2 < 10^{-6}$
- RRF $k = 60$; NREM slices: 50%/30%/20%; edge threshold $\tau = 0.35$; prune at $w < 0.05$
- GPU: RTX 4060 Ti 16GB, CUDA 13.2, cuSPARSE
- Dream daemon: 600 s idle / 50 new memories triggers

References

- [1] Aaron M Bornstein. Complementary learning systems for continual learning. *ICLR*, 2023.
- [2] Allan M Collins and Elizabeth F Loftus. A spreading activation theory of semantic processing. *Psychological Review*, 82(6):407–428, 1975.
- [3] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [4] Alex Graves et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016.
- [5] Dmitry Krotov and John J Hopfield. Dense associative memory for pattern recognition. *NeurIPS*, 2016.
- [6] Darren Edge Kumar et al. Graphrag: Unlocking llm discovery on narrative private data. *arXiv preprint*, 2024.
- [7] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.
- [8] Xi Victoria Lin et al. Hybrid retrieval-augmented generation: A survey. *arXiv preprint arXiv:2404.02455*, 2024.

- [9] James L McClelland and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- [10] Charles Packer et al. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2024.
- [11] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.
- [12] Mazemaker Project. Mazemaker benchmark invariants. <https://github.com/itsXactlY/mazemaker>, 2026. Internal benchmark protocol documentation.
- [13] Peter West et al. Memwalker: Guided tree search for long-context understanding. *arXiv preprint*, 2024.
- [14] Bohan Wu et al. Longmemeval: Evaluating long-term memory of large language models. In *International Conference on Learning Representations*, 2025.